# Frameworks: How-To Dungeon

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

# How-To Dungeon

In diesem Semester werden Sie im Praktikum schrittweise ein eigenes Rogue-like Computerspiel programmieren und dabei (hoffentlich) die Methoden aus der Vorlesung einsetzen können.

Damit Sie dabei nicht von Scratch starten müssen oder sich selbst in ein komplexes Spiele-Framework einarbeiten müssen, haben wir das Projekt "PM-Dungeon" ins Leben gerufen.

Der PM-Dungeon stellt wichtige Bausteine für das Spiel bereit, beispielsweise eine Game-Loop und eine API für das Generieren und Benutzen von Leveln und vieles andere mehr. Im Hintergrund werkelt das etablierte Open-Source-Spieleframework libGDX.

Wir werden uns in diesem How-To einen Überblick verschaffen und einen ersten Einstieg versuchen: Wir programmieren einen einfachen Helden.

# Projekt PM-Dungeon

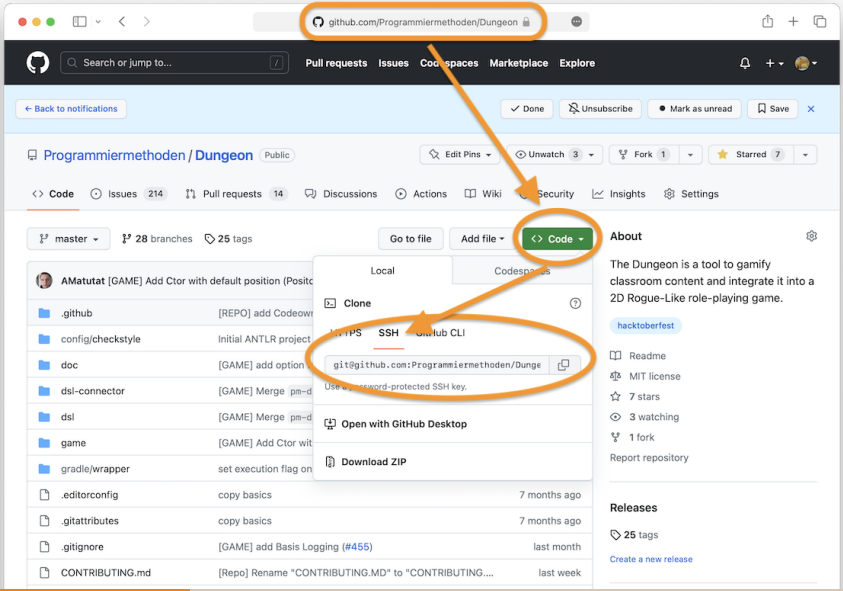
Das Projekt PM-Dungeon entstand in verschiedenen Forschungsprojekten und wurde (und wird) aktiv von Studierenden und wissenschaftlichen Mitarbeitern am Campus Minden entwickelt.

Aktuell läuft das Forschungsprojekt “Dungeon”, gefödert durch die Stiftung für Innovation in der Hochschullehre im “Freiraum 2022”. Dabei sollen diesmal nicht die Studierenden selbst Code schreiben, sondern die Lehrenden sollen Aufgaben in einer speziellen (von uns entwickelten) Programmiersprache schreiben (können), woraus dann ein fertiges Dungeon-Spiel generiert wird (mit der Aufgabe als Quest o.ä. im Dungeon eingebettet) und die Studierenden können durch das Spielen die Aufgaben lösen.

Sie werden merken, dass trotz klarer Richtlinien und Ideen die Entwicklung in der Praxis doch nicht so einfach ist und dass viele Dinge immer wieder geübt und erinnert werden müssen: Namen von Klassen und Methoden, sinnvolles Javadoc, Dokumentation jenseits des Javadoc, aber auch Commit-Messages und PR-Summaries.

# Installation des Frameworks

Sie finden das Projekt auf GitHub: https://github.com/Dungeon-CampusMinden/Dungeon.



# Java: Java SE 17 (LTS)

Wir benutzen im Dungeon-Projekt die aktuelle LTS-Version des JDK, d.h. **Java SE 17 (LTS)**. Sie können sich das JDK bei Oracle herunterladen oder Alternativen ausprobieren. Bitte unbedingt die jeweilige 64-bit Version nutzen!

In der Konsole sollte

```
java -version
```

ungefähr diese Ausgabe erzeugen (ignorieren Sie die Minor-Version, wichtig ist Major-Version: 17 bzw. "LTS"):

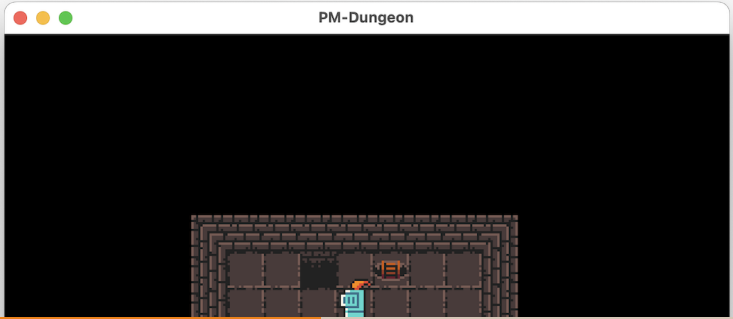
```
java version "17.0.6" 2023-01-17 LTS
Java(TM) SE Runtime Environment (build 17.0.6+9-LTS-190)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.6+9-LTS-190, mixed mode, sharing)
```

# Erster Test

Für einen ersten Test gehen Sie in der Konsole in den vorhin erzeugten neuen Ordner `pm-dungeon/` und führen Sie dort den Befehl

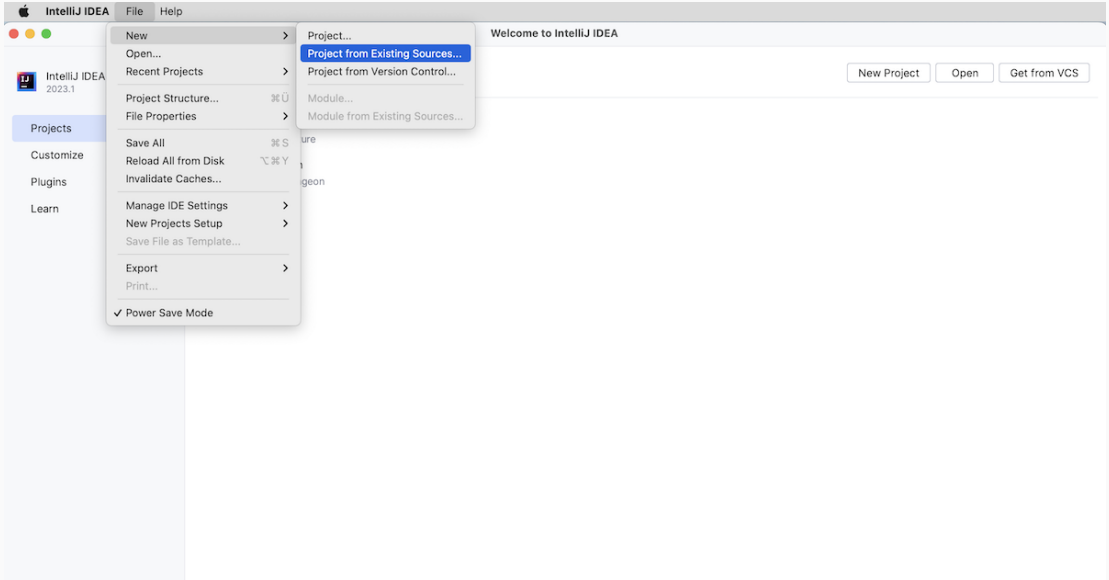
```
./gradlew run
```

aus. Dabei sollte das (mitgelieferte) Build-Tool Gradle starten und die benötigten Java-Bibliotheken herunterladen und schließlich das Spiel in einer Minimalversion starten - Sie sollten also ein Level sehen und einen hüpfenden Helden.



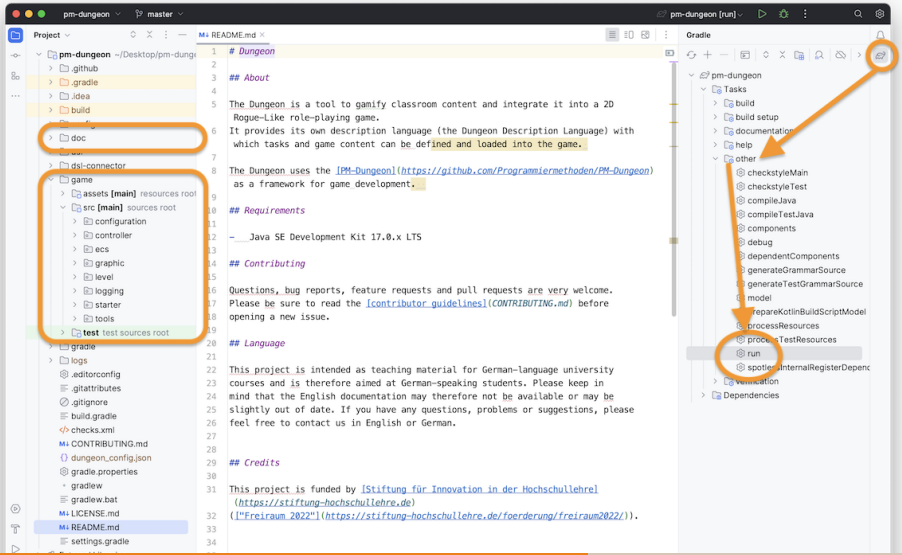
# Import in der IDE

Importieren Sie das Projekt als Gradle-basiertes Projekt, dann übernimmt die IDE die Konfiguration für Sie.



# Überblick über die Strukturen

Sie sehen im Package-Explorer eine Reihe von Unterprojekten. Für PM ist eigentlich nur "`game/`" relevant und "`doc/`" für die Dokumentation (die derzeit leider noch eine ziemliche Baustelle ist).



# Mein Held

Um einen besseren Blick in das System zu bekommen, erstellen wir einen eigenen einfachen Helden.

Dazu schalten wir zunächst in `starter.Game#setup()` in Zeile 118 die Initialisierung des Default-Helden ab (einfach die Zeile `hero = new Hero();` auskommentieren).

# Einschub: ECS oder Entities, Components und Systems

Der Held ist ein Element im Spiel. Dieses muss geeignet modelliert werden.

Unser Dungeon implementiert dabei eine Variante eines Entity Component System (*ECS*) und folgt damit "großen Vorbildern" wie beispielsweise Unity.

**Entity**

Die Idee dahinter ist: Alle Elemente im Spiel werden als *Entität* realisiert, d.h. der Held und die Monster und die Items, die man so finden kann, sind alles Entitäten. Im Prinzip könnten sogar die Boden- und Wandkacheln Entitäten sein.

Eine Entität an sich kann erst einmal nichts und dient nur als Container für *Components*. Das Spiel kennt alle derzeit vorhandenen Entitäten.

Unsere Basisklasse für Entitäten ist im Moment `ecs.entities.Entity`.

**Component**

Components bündeln bestimmte Werte einer Entität für bestimmte Zwecke, d.h. statt der Attribute in einer Klasse (Entität) nutzen wir hier eine weitere Kapselung.

Beispielsweise könnte man die Lebenspunkte u.ä. in einer `HealthComponent` verpacken und dann in einer Entität speichern. Oder man könnte in einer `VelocityComponent` hinterlegen, wie schnell eine

# Nun aber Helden!

## Ein Held ist eine Entität

Also legen wir eine neue Heldenklasse als Unterklasse von `ecs.entities.Entity` an:

```
public class MyHero extends Entity {}
```

Den nutzen wir in `starter.Game#setup()` in Zeile 118 an Stelle des vorhin auskommentierten Default-Helden: `hero = new MyHero();`.

Prinzipiell haben Sie damit alles, um das Spiel starten zu können. In der Praxis bekommen Sie aber eine Exception, weil die neue Entität vom Typ `MyHero` keine `PositionComponent` hat und das Spiel nicht sicher ist, wo es diese Entität nun anzeigen soll.

*Hinweis*: Eine Entität ist nur ein Container für Components. Insofern bräuchten wir eigentlich nicht von `Entity` ableiten, sondern könnten direkt eine Instanz von `Entity` anlegen und dort die gewünschten Components hineintun. In der Praxis mag es aber vielleicht ganz nett sein, trotzdem eine eigene Klasse zu spendieren, weil man hier Dinge für die Initialisierung kapseln kann - die würden sonst "frei" im `starter.Game#setup()` o.ä. "herumfliegen".

## Wo bin ich grad?

# Wrap-Up

Schauen Sie gern in die vorhandenen Klassen und Packages und in die Dokumentation hinein:

- Default-Held: `ecs.entities.Hero`
- Klassen in "`game/`"
- Dokumentation unter "`doc/`"

Anregungen für **Spielideen**

- Shattered Pixel Dungeon Rogue Beginners Guide Playthrough
- Shattered Pixel Dungeon Duelist Update!

# LICENSE

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.